k-Number	k2368701
Price by Monte Carlo	2.5903
Confidence interval for Price	(2.4141, 2.7665)
Delta by Monte Carlo	0.1916
Confidence interval for Delta	(0.1821, 0.2011)
Price by Finite Difference	2.5829
Delta by Finite Difference	0.1856

Introduction

This paper investigates the calculation of the price $V(t, S_t)$ and the delta of a European call option with strike price $K = 162.00$ and maturity $T = 1.70$ using two numerical methods: Monte Carlo and Finite Difference Methods. The objective of this paper is to assess which method provides the most accurate estimate of the price. The calculations presented in this paper are carried out using modified versions of the mathematical and computational techniques outlined in the lecture notes by John Armstrong, unless stated [Armstrong, 2024].

The analysis was conducted using the following setup. The initial stock price is $S_0 = 125.00$, and it is assumed that the stock price S_t evolves according to the stochastic differential equation (SDE):

$$dS_t = S_t \mu dt + S_t^{\gamma} \sigma(t)\, dW_t^{\mathbb{P}} \tag{1}$$

where $\mu = 0.1$ is the drift term, $\gamma = 0.98$ is an elasticity parameter that controls the volatility scaling with respect to the stock price, $\sigma(t)$ is the volatility function which varies over time, and $W_t^{\mathbb{P}}$ represents a standard Brownian motion under the real-world probability measure $\mathbb{P}$ that reflects the beliefs of the market about the future evolution of the stock price. The volatility function $\sigma(t)$ is defined deterministically as follows:

$$\sigma(t) = \begin{cases} 0.14 + 0.12 \times \frac{t-0.40}{T-0.40} & t > 0.40 \\ 0.14 & \text{otherwise} \end{cases}, \tag{2}$$

where t represents the time at which the stock price is evaluated and the volatility increases linearly after $t = 0.40$ until maturity T.

The derivative under consideration is a European call option, a financial contract that gives the holder the right, but not the obligation, to purchase the underlying asset for the specified strike price K at the maturity of the option T. The payoff of the option at maturity T and the stock price S_T is given by:

$$V(T, S_T) = \max(S_T - K, 0). \tag{3}$$

Computing the Partial Differential Equation

This paper begins by computing the partial differential equation (PDE), which establishes a fundamental link between the dynamics of the underlying asset

S_t, described in Equation (1), and the price of the option $V(t, S_t)$. The PDE describes how $V(t, S_t)$ evolves over time and with changes in the price of the underlying asset. The PDE is computed using a replication argument from the FM02 lecture series [Forde, 2024]. A trader begins with initial wealth X_0 and maintains a self-financing trading strategy in a market with no frictions. At each time $t \in [0, T]$, the trader dynamically adjusts their portfolio, holding Δ_t units of the stock S_t, where Δ_t is $\mathcal{F}_t^S$-adapted. The remaining wealth $X_t - \Delta_t S_t$ is placed in a risk-free bank account that accrues $(X_t - \Delta_t S_t) r dt$ in each infinitesimal time interval dt. Here, the risk-free rate r is the same for borrowing and saving. Under this setup, the trader's total wealth evolves according to:

$$\begin{aligned} dX_t &= \Delta_t dS_t + (X_t - \Delta_t S_t) r dt \\ &= \Delta_t \big(S_t \mu dt + S_t^\gamma \sigma(t)\, dW_t^{\mathbb{P}} \big) + (X_t - \Delta_t S_t) r dt \\ &= \big(r X_t + \Delta_t S_t (\mu - r) \big) dt + \Delta_t S_t^\gamma \sigma(t)\, dW_t^{\mathbb{P}}. \end{aligned} \tag{4}$$

Itô's Lemma provides the framework to derive the dynamics of the option price $V(t, S_t)$, which depends on both time and the stochastic process S_t. According to Itô's Lemma, the infinitesimal change in $V(t, S_t)$ is expressed as:

$$dV(t, S_t) = \frac{\partial V}{\partial t} dt + \frac{\partial V}{\partial S} dS_t + \frac{1}{2} \frac{\partial^2 V}{\partial S^2} (dS_t)^2. \tag{5}$$

Substituting dS_t and the quadratic variation $(dS_t)^2 = S_t^{2\gamma} \sigma^2(t) dt$. The dynamics of $V(t, S_t)$ are expressed as:

$$dV(t, S_t) = \left(\frac{\partial V}{\partial t} + \frac{\partial V}{\partial S} S_t \mu + \frac{1}{2} \frac{\partial^2 V}{\partial S^2} S_t^{2\gamma} \sigma(t)^2 \right) dt + \frac{\partial V}{\partial S} S_t^\gamma \sigma(t) dW_t^{\mathbb{P}}. \tag{6}$$

To ensure the self-financing trading strategy replicates the call option dynamics $X_t = V(t, S_t)$ until maturity where $X_T = V(T, S_T)$, the trader dynamically trades the stock holding $\Delta_t = \frac{\partial V}{\partial t}$ at each time t, while investing the remaining wealth in the risk free bank account. Here, $\frac{\partial V}{\partial t}$ is the delta of the option and measures the sensitivity of the derivative's price to changes in the price of the underlying asset. Equating the stochastic elements of Equation (4) and Equation (6), the drift terms must also match. This gives:

$$\frac{\partial V}{\partial t} + \frac{\partial V}{\partial S} S_t \mu + \frac{1}{2} \frac{\partial^2 V}{\partial S^2} S_t^{2\gamma} \sigma(t)^2 = rV + \frac{\partial V}{\partial t} S_t (\mu - r) \tag{7}$$

After canceling the μ terms, the resulting PDE is:

$$\frac{\partial V}{\partial t} + r S_t \frac{\partial V}{\partial S} + \frac{1}{2} S_t^{2\gamma} \sigma^2(t) \frac{\partial^2 V}{\partial S^2} = rV. \tag{8}$$

with the terminal condition given by Equation (3).

Interestingly, the resulting PDE and terminal condition are independent of μ, the drift rate of the underlying asset. Furthermore, the stochastic term is completely neutralized by the hedging process, resulting in a deterministic

PDE that governs the evolution of the option price. In the Black-Scholes (BS) model, the SDE has constant coefficients and admits a closed-form solution for European call options. Solving this PDE directly gives the option price for a given set of parameters. However, as the SDE in Equation (1) has non-constant coefficients, numerical methods must be employed to approximate the option price.

Monte Carlo Methods

The core mathematical foundation of Monte Carlo methods lies in the relationship between PDEs and stochastic processes. This connection is formalized by the Feynman-Kac theorem, which provides a probabilistic framework for solving a large class of PDEs by computing expectations under the risk-neutral measure $\mathbb{Q}$. The theorem states that the solution $V(t, S)$ to the PDE in Equation (8) can be represented as the expected discounted payoff of the option under the risk-neutral measure $\mathbb{Q}$. Specifically:

$$V(t, S) = \mathbb{E}^{\mathbb{Q}} \left[e^{-r(T-t)} \cdot \text{Payoff}(S_T) \,\middle|\, S_t = S \right] \tag{9}$$

Instead of directly solving the PDE, the Feynman-Kac theorem computes the option price by simulating the dynamics of the stock price S_t under $\mathbb{Q}$, and then averaging the discounted payoff across the simulated paths. The dynamics of S_t under $\mathbb{Q}$ are given by the SDE:

$$dS_t = S_t r dt + S_t^{\gamma} \sigma(t)\, dW_t^{\mathbb{Q}}. \tag{10}$$

Here, the risk-neutral measure $\mathbb{Q}$ is introduced solely to simplify pricing by ensuring that the discounted price process is a martingale. This property allows option prices to be computed as expectations under $\mathbb{Q}$. Importantly, under $\mathbb{Q}$, the drift term in Equation (1), is replaced by the risk-free rate r, reflecting the assumption that traders do not receive any risk premiums.

Euler-Maruyama is a numerical scheme used to discretize and simulate such SDEs. Applying this scheme, the solution of the SDE is defined as the limit of the following difference equation:

$$\delta S_t = S_t r\, \delta t + S_t^{\gamma} \sigma(t)\, \delta W_t, \quad \text{given } S_0, \tag{11}$$

as $\delta t \to 0$, where: $\delta S_t := S_{t+\delta t} - S_t$ and $\delta W_t := W_{t+\delta t} - W_t$. By the properties of Brownian motion, the increments $W_{t+\delta t} - W_t$ for points $t \in T$ are independent and normally distributed with mean zero and variance proportional to the time increment δt. Hence, it is possible to simulate $W_{t-\delta t} - W_t$ by $\sqrt{\delta t}\, \epsilon_t$ where $\epsilon_t \sim \mathcal{N}(0, 1)$ are independent standard normal random variables. In which case the Euler-Maruyama approximation is:

$$S_{t+\delta t} = S_t + r S_t \delta t + \sigma(t) S_t^{\gamma} \sqrt{\delta t}\, \epsilon_t. \tag{12}$$

As $\gamma = 0.98$ and $\sigma(t)$ is deterministic and smooth, the SDE satisfies Lipschitz conditions, ensuring a unique solution and the Euler-Maruyama scheme is well-defined [Forde, 2024].

Using Euler-Maruyama method to generate stock price paths, the option price can now be calculated. This involves simulating $N = 10,000$ stock price paths S_t starting at $S_0 = 125$, over 100 time steps, starting at $t = 0$ and extending to maturity at $T = 1.7$. Therefore, the time step $\delta t = \frac{T}{100}$. Then, the payoff of the call option is calculated for each path at maturity T according to Equation (3) where $\sigma(t)$ is calculated at each time step. Each payoff is discounted to it's present value and an average is computed across all paths:

$$V(0, S_0) \approx \frac{1}{N} \sum_{i=1}^{N} e^{-rT} \cdot \text{Payoff}(S_T^{(i)}). \tag{13}$$

The Delta of the option is approximated using the central difference method. The initial stock price, S_0, is perturbed slightly upward and downward by $h = S_0 \times 10^{-5}$. The estimated sensitivity of the option price to changes in the stock price is then given by:

$$\Delta \approx \frac{V(S_0 + h) - V(S_0 - h)}{2h}. \tag{14}$$

where $V(S_0 + h)$ and $V(S_0 - h)$ represent the option price when the stock price is increased and decreased by h, respectively.

The confidence intervals for the option price and delta are estimated using the Central Limit Theorem. The sample mean, μ_{sample} calculated in Equation (14), represents the average of the discounted payoffs, while the sample standard deviation is denoted by σ_{sample}. For a two-sided 95% confidence interval, the estimate is given by:

$$\mu_{\text{sample}} \pm \frac{1}{\sqrt{N}} Z\left(\frac{1 - \frac{95}{100}}{2}\right) \sigma_{\text{sample}}, \tag{15}$$

where $Z(\cdot)$ is the inverse cumulative distribution function of the standard normal distribution. This confidence interval quantifies the uncertainty in the Monte Carlo estimates, providing a range within which the true option price and delta are expected to lie with 95% confidence.

There are two types of error that result from the above calculations. The first is a statistical error that arises due to the finite number of paths, which introduces variability in the estimate. This can be reduced by increasing the number of paths N. This is the error that is computed by the confidence intervals and is inversely proportional to $\sqrt{N}$. The second error is the discretization error that arises from approximating the continuous SDE using a discrete time Euler-Maruyama scheme. This is not accounted for in the confidence intervals, but can be mitigated by refining the discretization. However, both improvements come at the cost of longer computational times.

Applying the methods described above, this paper estimates that the price of the call option is 2.5903 (95% CI : $2.4141, 2.7665$) and the delta is 0.1916

(95% CI : $0.1821, 0.2011$). However, this estimate can be improved by variance reduction. Antithetic sampling, generates $5,000$ independent standard normally distributed random numbers ϵ, alongside their antithetic pairs $-\epsilon$. This improves the accuracy by averaging the paired estimates. If the two paired estimates move in opposite directions, their average fluctuates less than the individual values, leading to a lower overall variance. However, due to the high strike price, only 13.64% of the simulated stock paths finish in the money. As a result, most of the payoffs were zero, leading to little variation between the paired values. This resulted in a weak negative covariance of -6.69. This method reduces the confidence interval by 32.31% but would be more effective in scenarios where the option is more likely to finish in the money, ensuring stronger payoff variation and a more negative covariance. This paper found the control variate method to be much better at reducing variance. The control variate method incorporates a related variable with a known expectation. Given a random variable M with unknown expectation $\mathbb{E}[M] = \mu$, introducing a second random variable T with a known expectation $\mathbb{E}[T] = \tau$ allows for an improved estimator $M^* = M + c(T - \tau)$. Since $\mathbb{E}[M^*] = \mathbb{E}[M]$, this adjustment maintains an unbiased estimate. The optimal coefficient c is chosen to minimize the variance:

$$\mathrm{Var}(M^*) = \mathrm{Var}(M) + c^2\mathrm{Var}(T) + 2c\mathrm{Cov}(M,T), \quad \text{where } c = -\frac{\mathrm{Cov}(M,T)}{\mathrm{Var}(T)}. \tag{16}$$

Substituting this into the variance equation results in:

$$\mathrm{Var}(M^*) = (1 - \rho^2)\mathrm{Var}(M), \tag{17}$$

where ρ is the correlation between M and T. As higher correlation leads to greater variance reduction, this method is extremely successful when T is chosen to be a call option under the BS SDE where $\rho = 0.95$. To ensure the error scales at the same order for all methods $n = 10,000$ scenarios are simulated for the pair of values (M, T). The control variate estimate for μ_{sample} is:

$$\mu_{\text{sample}} = -c\tau + \frac{1}{n}\sum_{i=1}^{n}(M_i + cT_i) \tag{18}$$

where τ is the Black-Scholes price. The price of the call option under the control variate method is estimated to be 2.5686 (95% CI : $2.4916, 2.6456$). Here, the confidence interval is reduced by 68.59%.

This paper tested the time-dependent volatility against expected values and the Monte Carlo method was carefully tested by fixing $\gamma = 1$ and maintaining constant volatility, with the resulting outputs compared to the closed-form solution of the BS model to ensure correct implementation.

Finite Difference Methods

This paper now computes the price of the call option by solving the PDE using Finite Difference Methods (FDM), numerical techniques used to solve differen-

tial equations by approximating derivatives with difference equations. These methods involve discretizing the continuous domain of the differential equation into a grid and then estimating the derivatives using central differences. The result is a large system of ordinary differential equations (ODEs) that approximate the original differential equation at discrete points on the grid. The ODE can then be solved using the Crank–Nicolson method. The price of the call option, $V(t, S)$, satisfies the PDE in Equation (8), with the terminal condition at maturity T given by the payoff of a European call option, in Equation (9).

The first step in solving the PDE requires discretizing its continuous domain by constructing a finite grid of evenly spaced points defined over the region $[0, T] \times [S_{\min}, S_{\max}]$. This grid enables the numerical approximation of $V(t, S)$ at each discrete time point. The grid consists of $\bar{N} = 1000$ steps in the time direction, with a time increment of $\delta t = \frac{T}{\bar{N}}$. The time points are numbered as $t_j = j\delta t$, where $j = 0, 1, \ldots, \bar{N}$. Similarly, the grid consists of $\bar{M} = 1000$ steps in the stock price or space direction. Numbering the stock price points as $S_{\min} = S_0, S_1, \ldots, S_{\bar{M}} = S_{\max}$, the stock price at the i-th point is defined as:

$$S_i = S_{\min} + i\delta S, \quad \text{where } \delta S = \frac{S_{\max} - S_{\min}}{\bar{M}}. \tag{19}$$

Thus, the total number of evenly spaced grid points in the discretized domain is $(\bar{N}+1) \times (\bar{M}+1)$. The choice of $\bar{N}$ and $\bar{M}$ sets the grid resolution and ensures a fair comparison with Monte Carlo based on computational points.

The next step is to impose appropriate boundary conditions at S_{min} and S_{max}. These conditions ensure the solution is well-defined and consistent with the financial interpretation at the grid edges. At the lower boundary, $S_{\min} = 0$ is chosen because the stock price cannot fall below zero. The value of the option at $S = 0$ is also zero, since it will not be exercised, giving:

$$V(t, S_{\min}) = 0, \quad \text{for all } t < T. \tag{20}$$

At the upper boundary, as $S_{\max} \to \infty$, the option's value asymptotically approaches its intrinsic value $(S - K)$, dominated by the stock price. For a call option, this behavior is captured by the boundary condition:

$$V(t, S_{\max}) = S_{\max} - Ke^{-r(T-t)} \quad \text{where} \quad S_{\max} = K\exp(8\sigma\sqrt{T}) \tag{21}$$

which ensures the upper boundary is sufficiently far so that the probability of the stock price exceeding $S_{\max}$ by maturity is negligible. This choice is based on the fact that the logarithm of the stock price is approximately normally distributed, and values beyond 8 standard deviations are highly unlikely. The boundaries of the grid are now fully defined.

Using finite-difference approximations, the PDE is discretized into a system of ODEs at each grid point S_i. Let $V_j^i := V(t_j, S_i)$ denote the option value at the i-th grid point in stock price space and the j-th time step. Using central difference approximations, the spatial derivatives in the PDE are approximated as follows. The first derivative of V with respect to S is:

$$\frac{\partial V}{\partial S}(t_j, S_i) \approx \frac{V(t_j, S_{i+1}) - V(t_j, S_{i-1})}{2\delta S} = \frac{V_j^{i+1} - V_j^{i-1}}{2\delta S}. \tag{22}$$

Similarly, for the second derivative:

$$\frac{\partial^2 V}{\partial S^2}(t_j, S_i) \approx \frac{V(t_j, S_{i+1}) - 2V(t_j, S_i) + V(t_j, S_{i-1})}{\delta S^2} = \frac{V_j^{i+1} - 2V_j^i + V_j^{i-1}}{\delta S^2}. \tag{23}$$

Substituting these approximations into the PDE allows us to approximate $V(t_j, S_i)$ at the grid points S_i by solving a system of $\bar{M} - 1$ ODEs for the values of V_j^i

$$\frac{\partial V_j^i}{\partial t} + \frac{\sigma^2(t) S_i^{2\gamma}}{\delta S^2}\left(V_j^{i+1} - 2V_j^i + V_j^{i-1}\right) + \frac{rS_i}{2\delta S}\left(V_j^{i+1} - V_j^{i-1}\right) - rV_j^i \approx 0, \tag{24}$$

for $i = 1, 2, \ldots, \bar{M} - 1$. This can be rewritten as:

$$\frac{\partial V_j^i}{\partial t} \approx a_i(t_j) V_j^{i-1} + b_i(t_j) V_j^i + c_i(t_j) V_j^{i+1}, \tag{25}$$

where:

$$a_i(t_j) = -\frac{\sigma^2(t) S_i^{2\gamma}}{2\delta S^2} + \frac{rS_i}{2\delta S}, \quad b_i(t_j) = r + \frac{\sigma^2(t) S_i^{2\gamma}}{\delta S^2}, \quad c_i(t_j) = -\frac{\sigma^2(t) S_i^{2\gamma}}{2\delta S^2} - \frac{rS_i}{2\delta S}. \tag{26}$$

Note that $a_i(t_j), b_i(t_j), c_i(t_j)$ all depend on t due to the time varying volatility. Therefore, it is important that volatility is calculated at each time point when solving the PDE. It is also essential to specify the boundary conditions, as the formula above is not valid at the top or bottom boundaries due to the inability to compute V_j^{i+1} and V_j^{i-1} respectively.

$$V_j^{\bar{M}} = S_{\max} - e^{-r(T-t)} K \quad \text{and} \quad V_j^0 = 0 \tag{27}$$

The system of ODEs can now be expressed in matrix form as follows:

$$\frac{d\mathbf{V}}{dt} \approx \Lambda(t_j)\mathbf{V}_j + \mathbf{W}_j \tag{28}$$

Where $\mathbf{V}_j$ is the vector containing components V_j^i for $i = 1, ..., \bar{M} - 1$, $\Lambda(t_j)$ is the tridiagonal matrix and the vector $\mathbf{W}_j$ which imposes the top and bottom boundary conditions is given:

$$\Lambda(t_j) = \begin{pmatrix} b_1 & c_1 & 0 & 0 & \cdots & 0 & 0 \\ a_2 & b_2 & c_2 & 0 & \cdots & 0 & 0 \\ 0 & a_3 & b_3 & c_3 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \cdots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & b_{\bar{M}-2} & c_{\bar{M}-2} \\ 0 & 0 & 0 & 0 & \cdots & a_{\bar{M}-1} & b_{\bar{M}-1} \end{pmatrix}, \quad \mathbf{W}_j = \begin{pmatrix} a_1 \cdot V_j^0 \\ 0 \\ \vdots \\ 0 \\ c_{\bar{M}-1} \cdot V_j^{\bar{M}} \end{pmatrix}. \tag{29}$$

Given that the option pricing problem has been approximated by a system of ODEs, the Crank–Nicolson scheme can now be derived to solve this system using central differences. The method begins with the finite-difference approximation:

$$\frac{\mathbf{V}_{j+1}^i - \mathbf{V}_j^i}{\delta t} \approx \frac{1}{2}\Lambda\left(\mathbf{V}_{j+1}^i + \mathbf{V}_j^i\right) + \frac{1}{2}\left(\mathbf{W}_{j+1} + \mathbf{W}_j\right), \tag{30}$$

Rearranging this equation, we obtain the Crank–Nicolson scheme:

$$\left(\mathbf{1} - \frac{1}{2}\Lambda\delta t\right)\mathbf{V}_{j+1}^{i} \approx \left(\mathbf{1} + \frac{1}{2}\Lambda\delta t\right)\mathbf{V}_{j}^{i} - \frac{1}{2}\delta t\left(\mathbf{W}_{j+1} + \mathbf{W}_{j}\right), \tag{31}$$

This linear equation is solved at each stage of the iteration, starting from the terminal condition at $t = T$ and stepping backwards in time to $t = 0$. As the problem is now in the form $Ax = b$, this paper employs scipy.sparse.linalg.spsolve which is designed to solve such equations for x efficiently in python. The price of the call option at S_0 is then calculated by interpolating the nearest grid points to S_0. This paper calculates the price of the option to be 2.5829 using FDM.

To approximate Δ, finite difference methods are applied. For interior grid points $i = 1, \ldots, \bar{M} - 1$, the central difference method is used:

$$\Delta_{i,j} \approx \frac{V_j^{i+1} - V_j^{i-1}}{2\delta S}. \tag{32}$$

At the boundaries, where central differences are not feasible, one-sided finite difference approximations are employed.

$$\Delta_{0,j} \approx \frac{V_j^{1} - V_j^{0}}{\delta S}, \quad \Delta_{M,j} \approx \frac{V_j^{M} - V_j^{M-1}}{\delta S}. \tag{33}$$

By applying the methods described above the delta is calculated to be 0.1856. This code was again tested by setting the time-varying volatility constant and $\gamma = 1$, then comparing with the closed-form BS solution.

Which Method is More Accurate?

This paper will assess the accuracy of Monte Carlo and FDM by examining convergence. A necessary condition for numerical accuracy, which refers to the rate at which a numerical solution approaches the true solution as computational resources increase. Stability, another necessary condition, is inherently satisfied by both methods and will not be investigated. As Crank-Nicolson is unconditionally stable, while Monte Carlo is robust to numerical instability.

Convergence is particularly important when comparing deterministic and stochastic numerical schemes. Unlike FDM which is deterministic and converges predictably as the grid size is refined, Monte Carlo which rely on random sampling, meaning a single realization may by chance be more accurate. Under the BS model with constant volatility and $\gamma = 1$ both Monte Carlo and FDM converge to a closed-form solution as the number of paths increases and as the grid is refined, respectively.

The convergence rate of Monte Carlo follows from the Central Limit Theorem, which states the sampling error decreases as the number of simulated paths, N, increases. To quantify this error, this paper calculates the Root Mean Square Error (RMSE), which measures the expected deviation of the Monte Carlo estimate from the true value:

$$\text{RMSE} = \sqrt{\mathbb{E}[(\mu_{\text{sample}} - \mu_{\text{true}})^2]} = \sqrt{\text{Var}(\mu_{\text{sample}})} = \frac{\sigma}{\sqrt{N}} \tag{34}$$

this paper justifies computing $\mu_{\text{true}} = 2.5850$ using the FDM price on a $20,000 \times 20,000$ grid, which accurately calculates the BS price to four decimal places. The RMSE helps reduce wild fluctuations in the estimated price by providing a measure of the deviation from the true value. Therefore, the Monte Carlo RMSE decreases at a rate of $\mathcal{O}(N^{-1/2})$. This means that to decrease the error by a factor of 10, the number of paths must be increased by a factor of 100. Since this convergence rate is rather slow, variance reduction techniques described previously can be used to improve efficiency. The RMSE for the control variate estimator is given by:

$$\text{RMSE} = \sqrt{\mathbb{E}[(\mu_{\text{sample}} - \mu_{\text{true}})^2]} = \sqrt{\text{Var}(\mu_{\text{sample}})} = \sqrt{\frac{\sigma^2(1-\rho^2)}{N}} = \frac{\sigma\sqrt{1-\rho^2}}{\sqrt{N}} \tag{35}$$

Since $0 \leq \rho^2 \leq 1$ the variance is reduced. However, the convergence rate remains $\mathcal{O}(N^{-1/2})$. As a result, a large number of samples are needed for high accuracy.

In Figure (1), this paper presents a log-log plot of the RMSE against the number of paths, comparing the BS Monte Carlo with the Monte Carlo and Control Variate methods for the SDE presented in this paper. A reference line representing the expected convergence rate of $\mathcal{O}(N^{-1/2})$ is also included. The results demonstrate that all three methods converge closely to this rate, verifying the theoretical convergence rate for the numerical scheme. Additionally, the time step remains constant as convergence is independent of δt for a reasonable δt. An additional step this paper has taken is to average the RMSE across multiple seeds to achieve a more robust estimate of the error, thereby reducing the impact of any random fluctuations that may arise from individual realizations of the stochastic process.

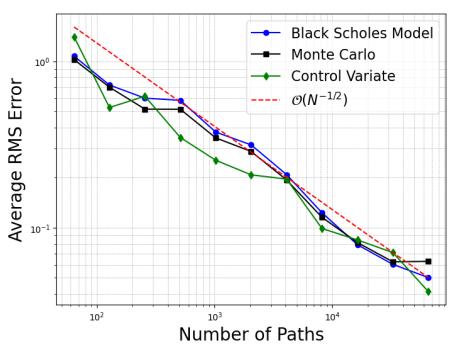


Figure 1: MC RMSE vs. Paths

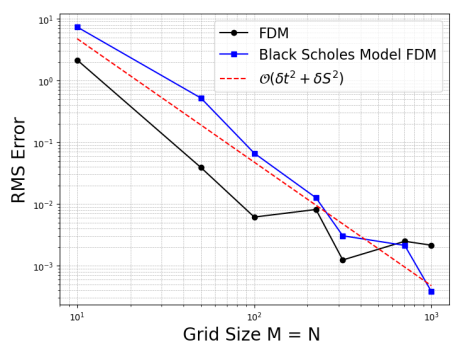


Figure 2: FDM RMSE vs. Grid Size

The convergence rate of the FDM is directly tied to the refinement of the grid on which the solution is computed. The Crank–Nicolson scheme, in particular, achieves a convergence rate of $\mathcal{O}(\delta t^2 + \delta S^2)$ [Qu and Liang, 2017]. As FDM is deterministic, the RMSE calculation simplifies to the computation of the absolute error. Figure (2), this paper presents a log-log plot of the RMSE against the grid refinement, comparing the BS FDM with the FDM for the SDE

presented in this paper. It was determined through methodical experimentation that both dimensions of the grid size needed to be increased equally to prevent a dominant error from affecting the results. Consequently, $\bar{N}$ was set equal to $\bar{M}$. Second-order convergence is verified by the reference line $\mathcal{O}(\delta t^2 + \delta S^2)$. As δt and δS are decreased the overall error decreases at a rate proportional to the square of these values. A potential source of error may be introduced by the interpolation function used to calculate the price at S_0 from the grid points, which is only first-order accurate. A potential improvement would be to design the grid in such a way that S_0 is always a grid point. However, it can be seen in Figure (2) that both FDM models appear to converge at the rate $\mathcal{O}(\delta t^2 + \delta S^2)$.

Since Monte Carlo and FDM converge to the same solution, this paper will consider the FDM which exhibits a faster convergence rate of $\mathcal{O}(\delta t^2 + \delta S^2)$ to be considered more accurate. A fast convergence rate means that the error decreases more rapidly as the number of iterations increases. Therefore, making the method more efficient and accurate overall. Furthermore, the deterministic nature of FDM is more favorable especially when the refinement of the numerical scheme is low. Figure (3), depicts the convergence to the true price for the Monte Carlo, Control Variate and FDM as the numerical scheme is refined. For a direct comparison between methods, this paper compares the number of points in the solution, where time steps are fixed and the number of paths increases, $100 \times N$. This is then compared to the grid size, $\bar{M}^2$. Figure (3), further justifies that FDM is the most accurate method, especially at a low refinement. It can also be seen that control variate significantly improves the Monte Carlo estimate. However, the control variate method runs into an issue for small sample sizes where the variance may be zero due to the high strike price.

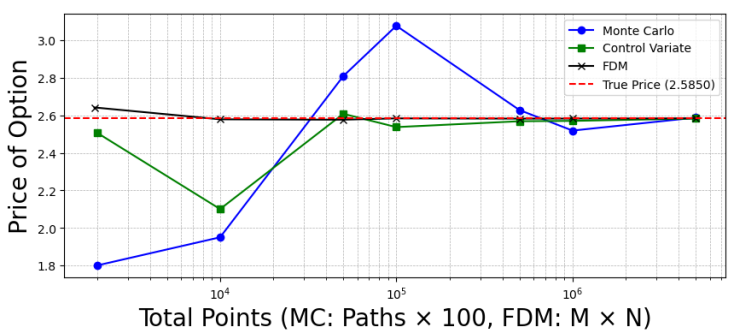


Figure 3: Monte Carlo vs. FDM Convergence

However, while this paper demonstrates that the FDM is more accurate for the assessed setup, the Monte Carlo method is better suited for high-dimensional problems where deterministic methods become impractical. This is primarily due to the curse of dimensionality in the FDM, which makes it computationally expensive for multi-asset or American options.

In conclusion, this paper finds that finite difference methods provide the most accurate estimate of the price for the setup described.

References

[Armstrong, 2024] Armstrong, J. (2024). 7ccmfm06 numerical and computational methods in finance.

[Forde, 2024] Forde, M. (2024). The black-scholes model. `https://keats.kcl.ac.uk/pluginfile.php/11910351/mod_resource/content/9/BlackScholesModel.pdf`. 7CCMFM02 Risk Neutral Valuation: Pricing and Hedging Derivatives.

[Qu and Liang, 2017] Qu, W. and Liang, Y. (2017). Stability and convergence of the crank-nicolson scheme for a class of variable-coefficient tempered fractional diffusion equations. *Advances in Difference Equations*, 2017(1).

Individual Question for Student with k-number k2368701

Suppose that a stock price S_t evolves according to the stochastic differential equation

$$dS_t = S_t \mu \, dt + S_t^{\gamma} \sigma(t) \, dW_t. \tag{36}$$

Here $S_0 = 125.00$, $\mu = 0.10$ and $\gamma = 0.98$ are constants, $\sigma(t)$ is given by the deterministic function

$$\sigma(t) = \begin{cases} 0.14 + 0.12 \times \frac{t-0.40}{T-0.40} & t > 0.40 \\ 0.14 & \text{otherwise} \end{cases},$$

and W_t is a Brownian motion. Suppose that there is also a risk-free asset which grows at the continuously compounded rate $r = 0.02$.

In this question you must calculate the price V of a call option with strike $K = 162.00$ and maturity $T = 1.70$.

Compute a partial differential equation that will be satsfied by V. Explain how knowing this partial differential equation allows you to compute the price of the option using a Monte Carlo method. Compute the price of the option and its delta using Monte Carlo methods with 10000 scenarios and 100 time-steps. Estimate a 95% confidence interval for the price and delta, you may ignore any error arising from the use of the Euler-Maruyama scheme in this calculation.

It is also possible to solve the partial differential equation using finite difference methods. Compute the price and delta of the option using the Crank–Nicolson method, stating in full the difference equations you have used for this computation.

You should explain which method you believe gives the most accurate estimate for the price and provide a justification for your answer which includes a plot illustrating your argument.

You must BEGIN your write up by displaying the following table of results.

k-Number	k2368701
Price by Monte Carlo	
Confidence interval for Price	
Delta by Monte Carlo	
Confidence interval for Delta	
Price by Finite Difference	
Delta by Finite Difference	

Kernel: Python 3 (system-wide)

Coursework Code

Initialization

In [3]:
```
import numpy as np
import matplotlib.pyplot as plt
import scipy.stats
import scipy.sparse
import scipy.sparse.linalg
from math import *
from scipy.stats import norm
```

Parameters

In [4]:
```
S0 = 125  # Initial stock price
K = 162   # Strike price
T = 1.7   # Time to maturity
r = 0.02  # Risk-free rate
gamma = 0.98 # Elasticity parameter
n_steps = 100 #Time Steps
n_paths = 10000 #Number of simulations
```

Time-Dependent Volatility Function

In [5]:
```
def sigma_t(t, T):
    if t > 0.40:
        return 0.14 + 0.12 * ((t-0.40) / (T-0.40))
    else:
        return 0.14
```

Testing the Time-Dependent Volatility Function

In [6]:
```
def test_sigma_t():
    T = 0.5
    t = np.linspace(0,T, 6) #[0.0  0.1   0.2   0.3   0.4   0.5 ]
    expected_values = [0.14,0.14,0.14, 0.14, 0.14, 0.14 + 0.12 * (0.5-
0.40) / (0.5-0.40)]
    for i in range(len(t)):
        t_value = t[i]
        expected = expected_values[i]
        computed = sigma_t(t_value, T)
        assert np.isclose(computed, expected, atol=1e-6)

test_sigma_t()
```

Average Volatility Function

In [7]:
```
def average_sigma_t(T, n_steps):
    times = np.linspace(0, T, n_steps)
    volatilities = [sigma_t(t, T) for t in times]
    avg_volatility = np.mean(volatilities)
    return avg_volatility

sigma = average_sigma_t(T, n_steps)
print(f"Average volatility: {sigma:.4f}")
```

Out[7]:
```
Average volatility: 0.1860
```

The Black-Scholes Model

In [8]:
```
def norm_cdf(x):
    return scipy.stats.norm.cdf(x)

def compute_d1_and_d2(S, t, K, T, r, sigma):
    tau = T - t
    d1 = 1 / (sigma * np.sqrt(tau)) * (np.log(S / K) + (r + 0.5 *
sigma**2) * tau)
    d2 = d1 - sigma * np.sqrt(tau)
    return d1, d2

def black_scholes_call_price(S, t, K, T, r, sigma):
    d1, d2 = compute_d1_and_d2(S, t, K, T, r, sigma)
    return S * norm_cdf(d1) - np.exp(-r * (T - t)) * K * norm_cdf(d2)

def black_scholes_call_delta(S, t, K, T, r, sigma):
    d1, _ = compute_d1_and_d2(S, t, K, T, r, sigma)
    return norm_cdf(d1)

bs_price = black_scholes_call_price(S0, 0, K, T, r, sigma)
bs_delta = black_scholes_call_delta(S0, 0, K, T, r, sigma)

print(f"Black-Scholes Call Price: {bs_price:.8f}")
print(f"Black-Scholes Call Delta: {bs_delta:.8f}")
```

Out[8]:
```
Black-Scholes Call Price: 3.21698775
Black-Scholes Call Delta: 0.20967593
```

Call Option

In [9]:
```
def payoff_function(S, K):
    return np.maximum(S - K, 0)
```

Monte Carlo Methods

In [10]:
```
np.random.seed(0)
random_numbers = np.random.randn(n_paths, n_steps)

def simulate_stock_paths(S0, r, sigma_t, T, n_steps, n_paths, gamma,
```

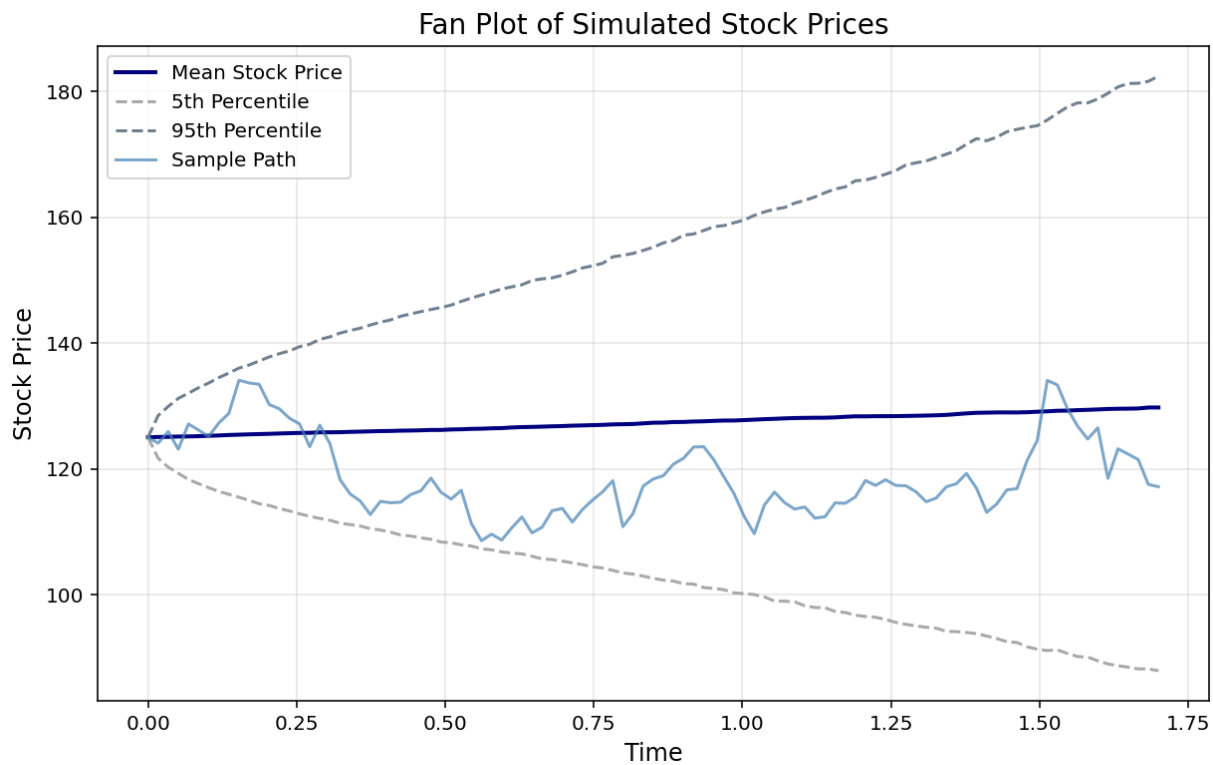
```
random_numbers):
    dt = T / n_steps
    times = np.linspace(0, T, n_steps + 1)
    S = np.zeros([n_paths, n_steps + 1])
    S[:, 0] = S0

    for i in range(n_steps):
        t = times[i]
        sigma = sigma_t(t, T)
        S[:, i + 1] = S[:, i] + S[:, i] * r * dt + S[:, i]**gamma *
sigma * np.sqrt(dt) * random_numbers[:, i]

    return S, times

def price_by_monte_carlo(S0, r, gamma, T, n_steps, n_paths, K,
payoff_function):
    S_twiddle, times = simulate_stock_paths(S0, r, sigma_t, T,
n_steps, n_paths, gamma, random_numbers)
    payoffs = payoff_function(S_twiddle[:, -1], K)
    p = 95
    alpha = norm.ppf((1 + p / 100) / 2)
    price = np.exp(-r * T) * np.mean(payoffs)
    sigma_sample = np.exp(-r * T) * np.std(payoffs)
    lower = price - alpha * sigma_sample / np.sqrt(n_paths)
    upper = price + alpha * sigma_sample / np.sqrt(n_paths)
    return lower, price, upper

def delta_by_monte_carlo(S0, r, gamma, T, n_steps, n_paths, K,
payoff_function):
    h = S0 * 1e-5
    S_twiddle, times = simulate_stock_paths(S0, r, sigma_t, T,
n_steps, n_paths, gamma, random_numbers)
    paths_for_S_plus_h = S_twiddle / S0 * (S0 + h)
    paths_for_S_minus_h = S_twiddle / S0 * (S0 - h)
    payoffs_S_plus_h = np.exp(-r * T) *
payoff_function(paths_for_S_plus_h[:, -1], K)
    payoffs_S_minus_h = np.exp(-r * T) *
payoff_function(paths_for_S_minus_h[:, -1], K)
    samples = (payoffs_S_plus_h - payoffs_S_minus_h) / (2 * h)
    p = 95
    alpha = norm.ppf((1 + p / 100) / 2)
    delta = np.mean(samples)
    sigma_sample = np.std(samples)
    lower = delta - alpha * sigma_sample / np.sqrt(n_paths)
    upper = delta + alpha * sigma_sample / np.sqrt(n_paths)
    return lower, delta, upper

lower_price, price, upper_price = price_by_monte_carlo(S0, r, gamma,
T, n_steps, n_paths, K, payoff_function)
lower_delta, delta, upper_delta = delta_by_monte_carlo(S0, r, gamma,
T, n_steps, n_paths, K, payoff_function)

print(f"Option Price: {price:.4f} (95% CI: {lower_price:.4f},
{upper_price:.4f})")
print(f"Delta: {delta:.4f} (95% CI: {lower_delta:.4f},
{upper_delta:.4f})")
```

Out[10]:
```
Option Price: 2.5903 (95% CI: 2.4141, 2.7665)
Delta: 0.1916 (95% CI: 0.1821, 0.2011)
```

Fan Plot (not in presentation)

In [11]:
```
def fan_plot(t, S):
    plt.figure(figsize=(10, 6))
    ax = plt.gca()
    ax.plot(t, np.mean(S, axis=0), label=f'Mean Stock Price',
linewidth=2, color='navy')
    ax.plot(t, np.percentile(S, 5, axis=0), label='5th Percentile',
linestyle='--', color='darkgrey')
    ax.plot(t, np.percentile(S, 95, axis=0), label='95th Percentile',
linestyle='--', color='slategrey')
    ax.plot(t, S[14, :], label='Sample Path', alpha=0.7,
color='steelblue')
    ax.set_xlabel('Time', fontsize=12)
    ax.set_ylabel('Stock Price', fontsize=12)
    ax.legend(fontsize=10, loc='upper left')
    ax.grid(alpha=0.3)
    plt.title('Fan Plot of Simulated Stock Prices', fontsize=14)
    plt.show()

S, times = simulate_stock_paths(S0, r, sigma_t, T, n_steps, n_paths,
gamma, random_numbers)
fan_plot(times, S)
```

Out[11]:



Testing the Monte Carlo

In [12]:
```
n_steps = 100
n_paths = 10000 # note can increase paths and decrease tolerance to
ensure this is more accurate

def test_price_and_delta():
    bs_price = black_scholes_call_price(S0, 0, K, T, r, sigma)
```

```
    bs_delta = black_scholes_call_delta(S0, 0, K, T, r, sigma)

    return bs_price, bs_delta

def test_simulate_stock_paths(S0, r, sigma, T, n_steps, n_paths,
gamma, random_numbers):
    dt = T / n_steps
    times = np.linspace(0, T, n_steps + 1)
    S = np.zeros([n_paths, n_steps + 1])
    S[:, 0] = S0

    for i in range(n_steps):
        S[:, i + 1] = S[:, i] + S[:, i] * r * dt + S[:, i]**gamma *
sigma * np.sqrt(dt) * random_numbers[:, i]

    return S, times

def test_price_by_monte_carlo(S0, r, gamma, T, n_steps, n_paths, K,
payoff_function):
    S_twiddle, times = test_simulate_stock_paths(S0, r, sigma, T,
n_steps, n_paths, gamma, random_numbers)
    payoffs = payoff_function(S_twiddle[:, -1], K)
    p = 95
    alpha = norm.ppf((1 + p / 100) / 2)
    price = np.exp(-r * T) * np.mean(payoffs)
    sigma_sample = np.exp(-r * T) * np.std(payoffs)
    lower = price - alpha * sigma_sample / np.sqrt(n_paths)
    upper = price + alpha * sigma_sample / np.sqrt(n_paths)
    return price

def test_delta_by_monte_carlo(S0, r, gamma, T, n_steps, n_paths, K,
payoff_function):
    h = S0 * 1e-5
    S_twiddle, times = test_simulate_stock_paths(S0, r, sigma, T,
n_steps, n_paths, gamma,random_numbers)
    paths_for_S_plus_h = S_twiddle / S0 * (S0 + h)
    paths_for_S_minus_h = S_twiddle / S0 * (S0 - h)
    payoffs_S_plus_h = np.exp(-r * T) *
payoff_function(paths_for_S_plus_h[:, -1], K)
    payoffs_S_minus_h = np.exp(-r * T) *
payoff_function(paths_for_S_minus_h[:, -1], K)
    samples = (payoffs_S_plus_h - payoffs_S_minus_h) / (2 * h)
    delta = np.mean(samples)
    return delta

bs_price, bs_delta = test_price_and_delta()
print(f"Black-Scholes Call Price: {bs_price:.4f}")
print(f"Black-Scholes Call Delta: {bs_delta:.4f}")

price = test_price_by_monte_carlo(S0, r, 1, T, n_steps, n_paths, K,
payoff_function)
delta = test_delta_by_monte_carlo(S0, r, 1, T, n_steps, n_paths, K,
payoff_function)

print(f"Option Price: {price:.4f}")
print(f"Delta: {delta:.4f}")

price_close = np.isclose(price, bs_price, rtol=0.05)
delta_close = np.isclose(delta, bs_delta, rtol=0.05)

print(f"Monte Carlo Price close to Black-Scholes? {price_close}")
```

```
print(f"Monte Carlo Delta close to Black-Scholes? {delta_close}")
```

Out[12]:
```
Black-Scholes Call Price: 3.2170
Black-Scholes Call Delta: 0.2097
Option Price: 3.2418
Delta: 0.2151
Monte Carlo Price close to Black-Scholes? True
Monte Carlo Delta close to Black-Scholes? True
```

In The Money %

In [13]:
```
def calculate_itm_probability(S0, r, sigma_t, T, n_steps, n_paths, K,
gamma):
    S_twiddle, times = simulate_stock_paths(S0, r, sigma_t, T,
n_steps, n_paths, gamma, random_numbers)
    S_T = S_twiddle[:, -1]
    itm_count = np.sum(S_T > K)
    itm_probability = (itm_count / n_paths) * 100
    return itm_probability

itm_probability = calculate_itm_probability(S0, r, sigma_t, T,
n_steps, n_paths, K, gamma)
print(f"Probability the stock finishes in the money:
{itm_probability:.2f}%")
```

Out[13]:
```
Probability the stock finishes in the money: 13.64%
```

In [14]:
```
def calculate_itm_black_scholes(S0, t, K, T, r, sigma):
    _, d2 = compute_d1_and_d2(S0, t, K, T, r, sigma)
    itm_probability = norm.cdf(d2) * 100
    return itm_probability

itm_probability_bs = calculate_itm_black_scholes(S0, 0, K, T, r,
sigma)
print(f"Probability the stock finishes in the money (Black-Scholes):
{itm_probability_bs:.2f}%")
```

Out[14]:
```
Probability the stock finishes in the money (Black-Scholes): 14.68%
```

Antithetic Sampling

In [15]:
```
np.random.seed(0)
n_paths_half = n_paths // 2
random_numbers = np.random.randn(n_paths_half, n_steps)

def anti_simulate_stock_paths(S0, r, sigma_t, T, n_steps,
n_paths_half, gamma, random_numbers):
    dt = T / n_steps
    times = np.linspace(0, T, n_steps + 1)
    S = np.zeros([n_paths_half, n_steps + 1])
    S[:, 0] = S0
    for i in range(n_steps):
        t = times[i]
        sigma = sigma_t(t, T)
        S[:, i + 1] = S[:, i] + S[:, i] * r * dt + S[:, i]**gamma *
```

```
sigma * np.sqrt(dt) * random_numbers[:, i]
    return S, times

def antithetic(S0, r, sigma_t, T, n_steps, n_paths_half, gamma, K,
payoff_function, random_numbers):
    minus_random_numbers = -random_numbers
    S_antithetic, _ = anti_simulate_stock_paths(S0, r, sigma_t, T,
n_steps, n_paths_half, gamma, random_numbers)
    S_antithetic_negative, _ = anti_simulate_stock_paths(S0, r,
sigma_t, T, n_steps, n_paths_half, gamma, minus_random_numbers)
    payoffs_antithetic = payoff_function(S_antithetic[:, -1], K)
    payoffs_antithetic_negative =
payoff_function(S_antithetic_negative[:, -1], K)
    combined_payoffs = 0.5 * (np.exp(-r * T) * payoffs_antithetic +
np.exp(-r * T) * payoffs_antithetic_negative)
    p = 95
    alpha = norm.ppf((1 + p / 100) / 2)
    price_antithetic = np.mean(combined_payoffs)
    sigma_antithetic = np.std(combined_payoffs)
    lower_antithetic = price_antithetic - alpha * sigma_antithetic /
np.sqrt(n_paths)
    upper_antithetic = price_antithetic + alpha * sigma_antithetic /
np.sqrt(n_paths)
    covariance = np.cov(payoffs_antithetic,
payoffs_antithetic_negative)[0, 1]
    return lower_antithetic, price_antithetic, upper_antithetic,
covariance

lower_antithetic, price_antithetic, upper_antithetic, covariance =
antithetic(S0, r, sigma_t, T, n_steps, n_paths_half, gamma, K,
payoff_function, random_numbers)
print(f"Price Estimate: {price_antithetic:.4f}")
print(f"95% Confidence Interval: ({lower_antithetic:.4f},
{upper_antithetic:.4f})")
print(f"Covariance: {covariance:.4f}")
```

Out[15]:
```
Price Estimate: 2.4481
95% Confidence Interval: (2.3288, 2.5674)
Covariance: -6.3759
```

In [16]:
```
interval_1 = upper_antithetic - lower_antithetic
interval_2 = upper_price - lower_price
comparison = ((interval_2 -interval_1)/interval_2)*100
print(f"Reduction: {comparison:.2f}%")
```

Out[16]:
```
Reduction: 32.31%
```

Control Variate

In [17]:
```
S0 = 125
K = 162
T = 1.7
r = 0.02
gamma = 0.98
n_steps = 100
n_paths = 20000
```

```
def simulate_gbm_paths(S0, r, sigma, T, n_steps, n_paths, gamma,
random_numbers):
    dt = T / n_steps
    times = np.linspace(0, T, n_steps + 1)
    S = np.zeros([n_paths, n_steps + 1])
    S[:, 0] = S0
    for i in range(n_steps):
        t = times[i]
        S[:, i + 1] = S[:, i] + S[:, i] * r * dt + S[:, i]**gamma *
sigma * np.sqrt(dt) * random_numbers[:, i]

    return S, times

def Control_Variate(S0, r, T, n_steps, n_paths, gamma,
random_numbers):
    n_paths_half = n_paths // 2
    S_SDE, times = simulate_stock_paths(S0, r, sigma_t, T, n_steps,
n_paths_half, gamma, random_numbers)
    S_GBM, times = simulate_gbm_paths(S0, r, sigma, T, n_steps,
n_paths_half, 1, random_numbers)

    payoff_SDE = payoff_function(S_SDE[:, -1], K)
    payoff_GBM = payoff_function(S_GBM[:, -1], K)

    BS_price = black_scholes_call_price(S0, 0, K, T, r, sigma)

    covTM = np.cov(np.exp(-r*T) * payoff_SDE, np.exp(-r*T) *
payoff_GBM)
    c = (-1 * covTM[0, 1] )/ covTM[1, 1]

    print("Correlation:", np.corrcoef(np.exp(-r*T) * payoff_SDE,
np.exp(-r*T) * payoff_GBM)[0, 1])

    VC_samples = np.exp(-r*T) * payoff_SDE + c * np.exp(-r*T) *
payoff_GBM - c * BS_price
    return VC_samples

np.random.seed(0)
n_paths_half = n_paths // 2
random_numbers = np.random.randn(n_paths_half, n_steps)

VC_samples = Control_Variate(S0, r, T, n_steps, n_paths, gamma,
random_numbers)

p = 95
alpha = norm.ppf((1 + p / 100) / 2)
price = np.mean(VC_samples)
sigma_control = np.std(VC_samples)
lower_control = price - alpha * sigma_control / np.sqrt(n_paths_half)
upper_control= price + alpha * sigma_control / np.sqrt(n_paths_half)

print(f"Price Estimate: {price:.4f}")
print(f"95% Confidence Interval: ({lower_control:.4f},
{upper_control:.4f})")
```

Out[17]:
```
Correlation: 0.9494038921040544
Price Estimate: 2.5705
95% Confidence Interval: (2.5152, 2.6259)
```

In [18]:
```
interval_1 = upper_control - lower_control
interval_2 = upper_price - lower_price
comparison = ((interval_2 -interval_1)/interval_2)*100
print(f"Reduction: {comparison:.2f}%")
n_paths = 10000
```

Out[18]: Reduction: 68.59%

Finite Difference Method

In [19]:
```
def bottom_boundary_condition(K, T, S_min, r, t):
    return np.zeros(t.shape)

def top_boundary_condition(K, T, S_max, r, t):
    return S_max - np.exp(-r * (T - t)) * K

def final_boundary_condition(K, T, S):
    return np.maximum(S - K, 0)

def sigma_t(t, T):
    return np.where(t > 0.40, 0.14 + 0.12 * (t - 0.40) / (T - 0.40),
0.14)

# note different sigma function to test

def compute_abc(K, T, t, sigma_t, r, S, dt, dS, gamma):
    sigma = sigma_t(t, T)
    a = -sigma**2 * S**(2 * gamma) / (2 * dS**2) + r * S / (2 * dS)
    b = r + sigma**2 * S**(2 * gamma) / (dS**2)
    c = -sigma**2 * S**(2 * gamma) / (2 * dS**2) - r * S / (2 * dS)
    return a, b, c

def compute_lambda(a, b, c):
    return scipy.sparse.diags([a[1:], b, c[:-1]], offsets=[-1, 0, 1],
format='csr')

def compute_W(a, b, c, V0, VM):
    W = np.zeros(len(b))
    W[0] = a[0] * V0
    W[-1] = c[-1] * VM
    return W

def compute_delta(V, dS):
    N, M_plus_1 = V.shape
    M = M_plus_1 - 1
    Delta = np.zeros_like(V)

    Delta[:, 1:M] = (V[:, 2:M+1] - V[:, 0:M-1]) / (2 * dS)

    Delta[:, 0] = (V[:, 1] - V[:, 0]) / dS
    Delta[:, M] = (V[:, M] - V[:, M-1]) / dS

    return Delta

def price_call_crank_nicolson(K, T, r, sigma_t, N, M, gamma, S0):
    dt = T / N
    S_min = 0
    S_max = K * np.exp(8 * sigma_t(0, T) * np.sqrt(T))
```

```
    dS = (S_max - S_min) / M
    S = np.linspace(S_min, S_max, M + 1)
    t = np.linspace(0, T, N + 1)
    V = np.zeros((N + 1, M + 1))


    V[:, -1] = top_boundary_condition(K, T, S_max, r, t)
    V[:, 0] = bottom_boundary_condition(K, T, S_min, r, t)
    V[-1, :] = final_boundary_condition(K, T, S)

    for i in range(N - 1, -1, -1):
        a, b, c = compute_abc(K, T, t[i], sigma_t, r, S[1:-1], dt, dS,
gamma)
        Lambda = compute_lambda(a, b, c)
        identity = scipy.sparse.identity(M - 1)

        Wt = compute_W(a, b, c, V[i, 0], V[i, M])
        Wt_plus_dt = compute_W(a, b, c, V[i + 1, 0], V[i + 1, M])

        V[i, 1:M] = scipy.sparse.linalg.spsolve(
            identity + 0.5 * Lambda * dt,
            (identity - 0.5 * Lambda * dt).dot(V[i + 1, 1:M]) - 0.5 *
dt * (Wt_plus_dt + Wt)
        )

    Delta = compute_delta(V, dS)

    price_at_S0 = np.interp(S0, S, V[0, :])
    delta_at_S0 = np.interp(S0, S, Delta[0, :])

    return price_at_S0, delta_at_S0

gamma = 0.98
K = 162
T = 1.7
r = 0.02
N = 1000
M = 1000
S0 = 125

price_at_t0, delta_at_t0 = price_call_crank_nicolson(K, T, r, sigma_t,
N, M, gamma, S0)

print(f"Option Price at t=0 and S={S0}: {price_at_t0:.4f}")
print(f"Delta at t=0 and S={S0}: {delta_at_t0:.4f}")
```

Out[19]:
```
Option Price at t=0 and S=125: 2.5829
Delta at t=0 and S=125: 0.1856
```

High Accuracy Estimate

In [20]:
```
#gamma = 0.98
#K = 162
#T = 1.7
#r = 0.02
#N = 20000
#M = 20000
#S0 = 125
#price_at_t0, delta_at_t0 = price_call_crank_nicolson(K, T, r,
```

```
sigma_t, N, M, gamma, S0)
#print(f"Option Price at t=0 and S={S0}: {price_at_t0:.4f}")
#print(f"Delta at t=0 and S={S0}: {delta_at_t0:.4f}")
```

In [21]:

```
price_estimate = 2.5850
```

Testing Finite Difference Method

In [22]:

```
def sigma(t, T):
    return 0.186

gamma = 1

price_at_t0, delta_at_t0 = price_call_crank_nicolson(K, T, r, sigma,
N, M, gamma, S0)

print(f"Option Price at t=0 and S={S0}: {price_at_t0:.4f}")
print(f"Delta at t=0 and S={S0}: {delta_at_t0:.4f}")

price_close = np.isclose(price_at_t0, bs_price, rtol=0.05)
delta_close = np.isclose(delta_at_t0, bs_delta, rtol=0.05)

print(f"Monte Carlo Price close to Black-Scholes? {price_close}")
print(f"Monte Carlo Delta close to Black-Scholes? {delta_close}")
```

Out[22]:

```
Option Price at t=0 and S=125: 3.2166
Delta at t=0 and S=125: 0.2097
Monte Carlo Price close to Black-Scholes? True
Monte Carlo Delta close to Black-Scholes? True
```

Prove Accuracy to 4dp

In [23]:

```
#gamma = 1
#K = 162
#T = 1.7
#r = 0.02
#N = 20000
#M = 20000
#S0 = 125
#price_at_t0, delta_at_t0 = price_call_crank_nicolson(K, T, r, sigma,
N, M, gamma, S0)
#print(f"Option Price at t=0 and S={S0}: {price_at_t0:.8f}")
#print(f"Delta at t=0 and S={S0}: {delta_at_t0:.8f}")
#print(f"BS Price: {bs_price:.8f}")
#print(f"BS Delta: {bs-delta:.8f}")
```

MC Convergence

In [24]:

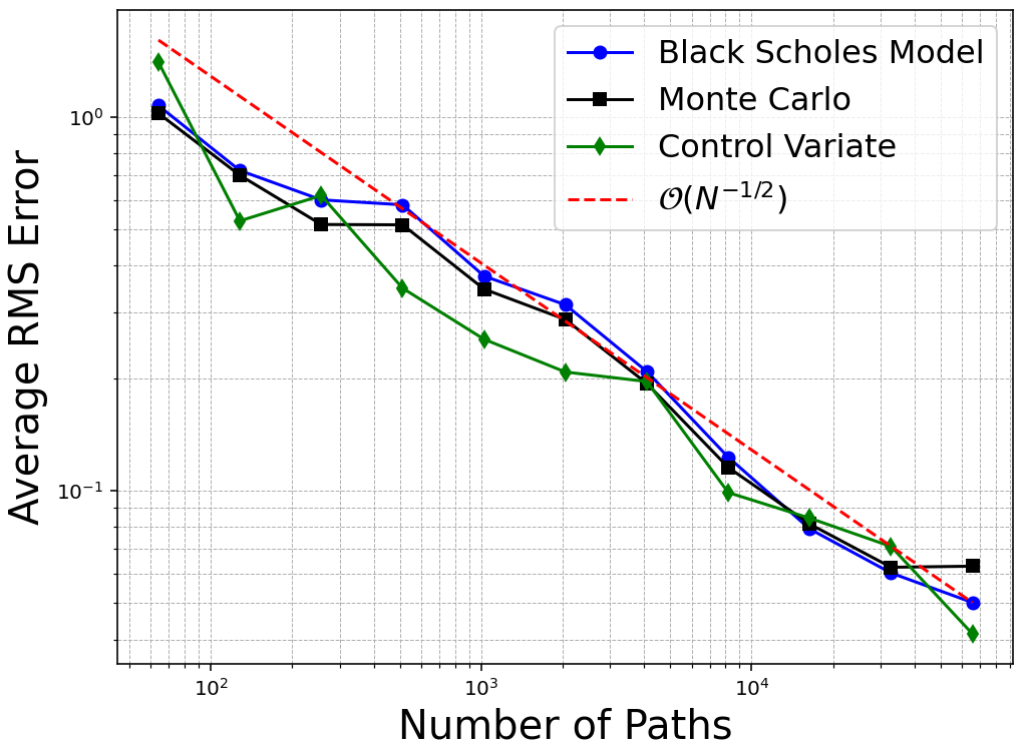
```
def simulate_gbm_paths(S0, r, sigma, T, n_steps, n_paths, seed=None):
    if seed is not None:
        np.random.seed(seed)
    gamma = 1
    dt = T / n_steps
```

```
    times = np.linspace(0, T, n_steps + 1)
    random_numbers = np.random.randn(n_paths, n_steps)
    S = np.zeros([n_paths, n_steps + 1])
    S[:, 0] = S0
    for i in range(n_steps):
        S[:, i + 1] = S[:, i] + S[:, i] * r * dt + S[:, i]**gamma *
sigma * np.sqrt(dt) * random_numbers[:, i]
    return S, times

def price_by_monte_carlo(S0, r, sigma, T, n_steps, n_paths, K,
payoff_function, seed=None):
    drift = r
    S_paths, times = simulate_gbm_paths(S0, drift, sigma, T, n_steps,
n_paths, seed=seed)
    payoffs = payoff_function(S_paths[:, -1], K)
    price = np.exp(-r * T) * np.mean(payoffs)
    return price

def simulate_stock_paths(S0, r, sigma_t, T, n_steps, n_paths, gamma,
seed=None):
    if seed is not None:
        np.random.seed(seed)
    gamma = 0.98
    dt = T / n_steps
    times = np.linspace(0, T, n_steps + 1)
    random_numbers = np.random.randn(n_paths, n_steps)
    S = np.zeros([n_paths, n_steps + 1])
    S[:, 0] = S0

    for i in range(n_steps):
        t = times[i]
        sigma = sigma_t(t, T)
        S[:, i + 1] = S[:, i] + S[:, i] * r * dt + S[:, i]**gamma *
sigma * np.sqrt(dt) * random_numbers[:, i]

    return S, times

def price_by_monte_carlo_sde(S0, r, gamma, T, n_steps, n_paths, K,
payoff_function, sigma_t, seed=None):
    random_numbers = np.random.randn(n_paths, n_steps)
    S_paths, times = simulate_stock_paths(S0, r, sigma_t, T, n_steps,
n_paths, gamma, seed=seed)
    payoffs = payoff_function(S_paths[:, -1], K)
    price = np.exp(-r * T) * np.mean(payoffs)
    return price


def price_by_monte_carlo_cv(S0, r, gamma, T, n_steps, n_paths, K,
payoff_function, sigma_t, seed=None):
    S_GBM, times = simulate_gbm_paths(S0, r, sigma, T, n_steps,
n_paths, seed=None)
    S_SDE, times = simulate_stock_paths(S0, r, sigma_t, T, n_steps,
n_paths, gamma, seed=None)
    payoff_SDE = payoff_function(S_SDE[:, -1], K)
    payoff_GBM = payoff_function(S_GBM[:, -1], K)
    BS_price = black_scholes_call_price(S0, 0, K, T, r, sigma)
    covTM = np.cov(np.exp(-r*T) * payoff_SDE, np.exp(-r*T) *
payoff_GBM)
    c = (-1 * covTM[0, 1] )/ covTM[1, 1]
    VC_samples = np.exp(-r*T) * payoff_SDE + c * np.exp(-r*T) *
payoff_GBM - c * BS_price
```

```
    price = np.mean(VC_samples)

    return price

def payoff_function(S, K):
    return np.maximum(S - K, 0)

S0 = 125
K = 162
T = 1.7
r = 0.02
sigma = 0.186
n_steps = 100
n_seeds = 10

def sigma_t(t, T):
    if t > 0.40:
        return 0.14 + 0.12 * ((t-0.40) / (T-0.40))
    else:
        return 0.14

analytical_price = black_scholes_call_price(S0, 0, K, T, r, sigma)
alternative_price_estimate = 2.5850

n_points = 11
path_sizes = [2**i for i in range(6, 6 + n_points)]
average_rms_error_gbm = np.zeros(len(path_sizes))
average_rms_error_sde = np.zeros(len(path_sizes))
average_rms_error_cv = np.zeros(len(path_sizes))

for i, n_paths in enumerate(path_sizes):
    seed_rms_errors_gbm = []
    seed_rms_errors_sde = []
    seed_rms_errors_cv = []

    for seed in range(n_seeds):
        mc_price_gbm = price_by_monte_carlo(S0, r, sigma, T, n_steps,
n_paths, K, payoff_function, seed=seed)
        error_gbm = mc_price_gbm - analytical_price
        seed_rms_errors_gbm.append(error_gbm**2)

        gamma = 0.98

        mc_price_sde = price_by_monte_carlo_sde(S0, r, gamma, T,
n_steps, n_paths, K, payoff_function, sigma_t, seed=seed)
        error_sde = mc_price_sde - alternative_price_estimate
        seed_rms_errors_sde.append(error_sde**2)

        mc_price_cv = price_by_monte_carlo_cv(S0, r, gamma, T,
n_steps, n_paths, K, payoff_function, sigma_t, seed=seed)
        error_cv = mc_price_cv- alternative_price_estimate
        seed_rms_errors_cv.append(error_cv**2)

    average_rms_error_gbm[i] = sqrt(np.mean(seed_rms_errors_gbm))
    average_rms_error_sde[i] = sqrt(np.mean(seed_rms_errors_sde))
    average_rms_error_cv[i] = sqrt(np.mean(seed_rms_errors_cv))

plt.figure(figsize=(8, 6))
plt.loglog(path_sizes, average_rms_error_gbm, marker='o', linestyle='-
', label="Black Scholes Model", color='blue')
plt.loglog(path_sizes, average_rms_error_sde, marker='s',linestyle='-
```

```
', label="Monte Carlo", color='black')
plt.loglog(path_sizes, average_rms_error_cv, marker='d',linestyle='-',
label="Control Variate", color='green')


convergence_line = (average_rms_error_gbm[-1] / path_sizes[-1]**
(-1/2)) * np.array(path_sizes)**(-1/2)
plt.loglog(path_sizes, convergence_line, linestyle='--', color='red',
label=r"$\mathcal{O}(N^{-1/2})$")

plt.xlabel('Number of Paths' , fontsize=20)
plt.ylabel('Average RMS Error', fontsize=20)
plt.grid(True, which="both", linestyle="--", linewidth=0.5)
plt.legend(fontsize=16)
plt.show()
```

Out[24]:



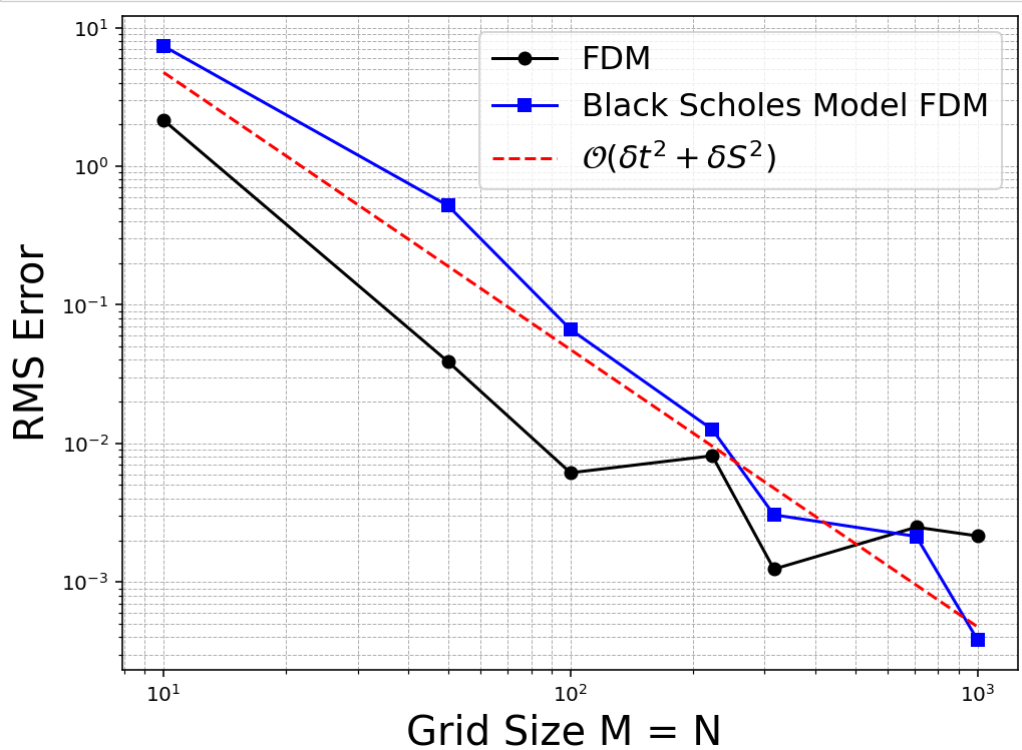
In [25]:

```
gamma = 0.98
K = 162  # Strike price
T = 1.7  # Time to maturity
r = 0.02  # Risk-free interest rate
S0 = 125  # Initial spot price

def bottom_boundary_condition(K, T, S_min, r, t):
    return np.zeros(t.shape)

def top_boundary_condition(K, T, S_max, r, t):
    return S_max - np.exp(-r * (T - t)) * K

def final_boundary_condition(K, T, S):
    return np.maximum(S - K, 0)

def sigma_t(t, T):
    return np.where(t > 0.40, 0.14 + 0.12 * (t - 0.40) / (T - 0.40),
0.14)

def compute_abc(K, T, t, sigma_t, r, S, dt, dS, gamma):
```

```
    sigma = sigma_t(t, T)
    a = -sigma**2 * S**(2 * gamma) / (2 * dS**2) + r * S / (2 * dS)
    b = r + sigma**2 * S**(2 * gamma) / (dS**2)
    c = -sigma**2 * S**(2 * gamma) / (2 * dS**2) - r * S / (2 * dS)
    return a, b, c

def compute_lambda(a, b, c):
    return scipy.sparse.diags([a[1:], b, c[:-1]], offsets=[-1, 0, 1],
format='csr')

def compute_W(a, b, c, V0, VM):
    W = np.zeros(len(b))
    W[0] = a[0] * V0
    W[-1] = c[-1] * VM
    return W

def price_call_crank_nicolson(K, T, r, sigma_t, N, M, gamma, S0):
    dt = T / N
    S_min = 0
    S_max = K * np.exp(8 * sigma_t(0, T) * np.sqrt(T))
    dS = (S_max - S_min) / M
    S = np.linspace(S_min, S_max, M + 1)
    t = np.linspace(0, T, N + 1)
    V = np.zeros((N + 1, M + 1))

    V[:, -1] = top_boundary_condition(K, T, S_max, r, t)
    V[:, 0] = bottom_boundary_condition(K, T, S_min, r, t)
    V[-1, :] = final_boundary_condition(K, T, S)

    for i in range(N - 1, -1, -1):
        a, b, c = compute_abc(K, T, t[i], sigma_t, r, S[1:-1], dt, dS,
gamma)
        Lambda = compute_lambda(a, b, c)
        identity = scipy.sparse.identity(M - 1)

        Wt = compute_W(a, b, c, V[i, 0], V[i, M])
        Wt_plus_dt = compute_W(a, b, c, V[i + 1, 0], V[i + 1, M])

        V[i, 1:M] = scipy.sparse.linalg.spsolve(
            identity + 0.5 * Lambda * dt,
            (identity - 0.5 * Lambda * dt).dot(V[i + 1, 1:M]) - 0.5 *
dt * (Wt_plus_dt + Wt)
        )

    price_at_S0 = np.interp(S0, S, V[0, :])


    return price_at_S0

def sigma(t, T):
    return 0.186

gamma = 0.98
K = 162
T = 1.7
r = 0.02
S0 = 125
M_list = np.array([10, 50, 100, 223, 316, 707, 1000])
analytical_price = 2.5850
analytical_price2 = 3.2170
```

```
error_crank = []
error_crank2 = []

for M, N in zip(M_list, M_list):
    price_cn = price_call_crank_nicolson(K, T, r, sigma_t, N, M,
gamma, S0)

    price_cn2 = price_call_crank_nicolson(K, T, r, sigma, N, M, 1, S0)

    error = abs(analytical_price - price_cn)
    error2 = abs(analytical_price2 - price_cn2)
    error_crank.append(error)
    error_crank2.append(error2)


ref_M = np.array([M_list[0], M_list[6]])
ref_error = 0.5 * (error_crank[0] + error_crank2[0]) * (ref_M /
M_list[0]) ** -2

plt.figure(figsize=(8, 6))
plt.loglog(M_list, error_crank, marker='o', color='black',
linestyle='-', label="FDM")
plt.loglog(M_list, error_crank2, marker='s', color='blue',
linestyle='-', label="Black Scholes Model FDM")
plt.loglog(ref_M, ref_error, linestyle='--', color='r',
label=r"$\mathcal{O}(\delta t^2 + \delta S^2)$")
plt.xlabel("Grid Size M = N ", fontsize=20)
plt.ylabel("RMS Error", fontsize=20)
plt.legend(fontsize=16)
plt.grid(True, which="both", linestyle="--", linewidth=0.5)
plt.show()
```

Out[25]:



In [26]:

```
S0 = 125
K = 162
T = 1.7
r = 0.02
```

```
gamma = 0.98
n_steps = 100

def simulate_stock_paths(S0, r, sigma_t, T, n_steps, n_paths, gamma,
random_numbers):
    dt = T / n_steps
    times = np.linspace(0, T, n_steps + 1)
    S = np.zeros([n_paths, n_steps + 1])
    S[:, 0] = S0

    for i in range(n_steps):
        t = times[i]
        sigma = sigma_t(t, T)
        S[:, i + 1] = S[:, i] + S[:, i] * r * dt + S[:, i]**gamma *
sigma * np.sqrt(dt) * random_numbers[:, i]

    return S, times

def price_by_monte_carlo(S0, r, gamma, T, n_steps, n_paths, K,
payoff_function):
    random_numbers = np.random.randn(n_paths, n_steps)
    S_twiddle, times = simulate_stock_paths(S0, r, sigma_t, T,
n_steps, n_paths, gamma, random_numbers)
    payoffs = payoff_function(S_twiddle[:, -1], K)

    price = np.exp(-r * T) * np.mean(payoffs)
    return price
num_paths_list = np.array([20, 100, 500, 1000, 5000, 10000, 50000])

prices = []
for n_paths in num_paths_list:
    price = price_by_monte_carlo(S0, r, gamma, T, n_steps, n_paths, K,
payoff_function)
    prices.append(price)

x_values = num_paths_list * 100
```

In [27]:

```
def simulate_stock_paths(S0, r, sigma_t, T, n_steps, n_paths, gamma,
random_numbers):
    dt = T / n_steps
    times = np.linspace(0, T, n_steps + 1)
    S = np.zeros([n_paths, n_steps + 1])
    S[:, 0] = S0

    for i in range(n_steps):
        t = times[i]
        sigma = sigma_t(t, T)
        S[:, i + 1] = S[:, i] + S[:, i] * r * dt + S[:, i]**gamma *
sigma * np.sqrt(dt) * random_numbers[:, i]

    return S, times

def average_sigma_t(T, n_steps):
    times = np.linspace(0, T, n_steps)
    volatilities = [sigma_t(t, T) for t in times]
    avg_volatility = np.mean(volatilities)
    return avg_volatility

sigma = average_sigma_t(T, n_steps)
```

```
def simulate_gbm_paths(S0, r, sigma, T, n_steps, n_paths, gamma,
random_numbers):
    dt = T / n_steps
    times = np.linspace(0, T, n_steps + 1)
    S = np.zeros([n_paths, n_steps + 1])
    S[:, 0] = S0
    for i in range(n_steps):
        t = times[i]
        S[:, i + 1] = S[:, i] + S[:, i] * r * dt + S[:, i]**gamma *
sigma * np.sqrt(dt) * random_numbers[:, i]

    return S, times

def Control_Variate(S0, r, T, n_steps, n_paths, gamma,
random_numbers):
    S_SDE, times = simulate_stock_paths(S0, r, sigma_t, T, n_steps,
n_paths, gamma, random_numbers)
    S_GBM, times = simulate_gbm_paths(S0, r, sigma, T, n_steps,
n_paths, 1, random_numbers)

    payoff_SDE = payoff_function(S_SDE[:, -1], K)
    payoff_GBM = payoff_function(S_GBM[:, -1], K)

    BS_price = black_scholes_call_price(S0, 0, K, T, r, sigma)

    covTM = np.cov(np.exp(-r*T) * payoff_SDE, np.exp(-r*T) *
payoff_GBM)
    c = (-1 * covTM[0, 1] )/ covTM[1, 1]

    VC_samples = np.exp(-r*T) * payoff_SDE + c * np.exp(-r*T) *
payoff_GBM - c * BS_price
    return VC_samples

num_paths_list = np.array([20, 100, 500, 1000, 5000, 10000, 50000])
prices_cv = []
for n_paths in num_paths_list:
    np.random.seed(0)
    random_numbers = np.random.randn(n_paths, n_steps)
    VC_samples = Control_Variate(S0, r, T, n_steps, n_paths, gamma,
random_numbers)
    price = np.mean(VC_samples)
    prices_cv.append(price)
```

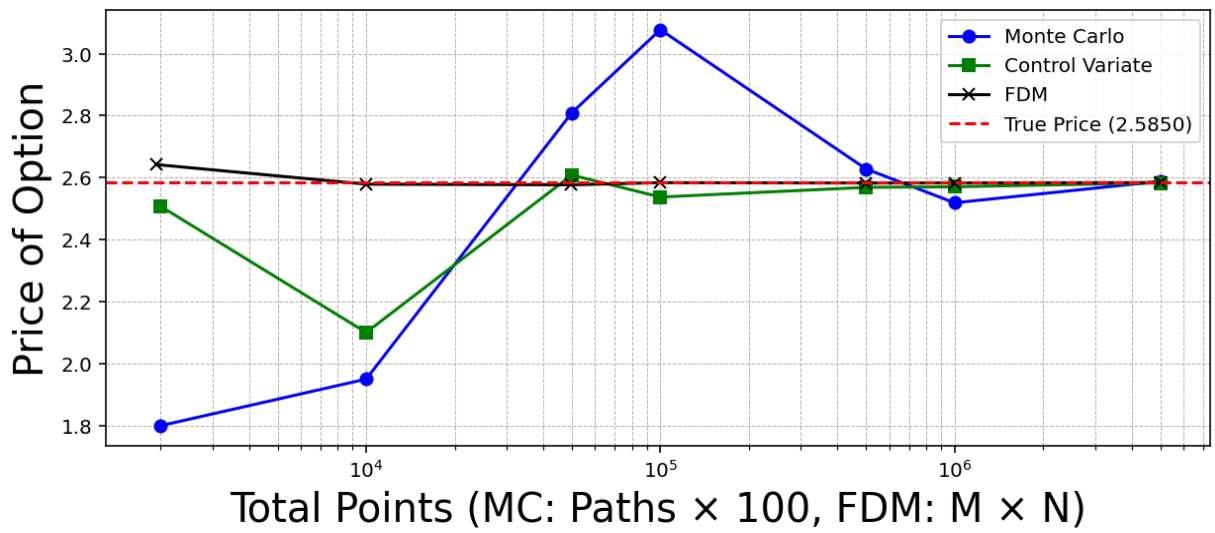
In [28]:

```
def bottom_boundary_condition(K, T, S_min, r, t):
    return np.zeros(t.shape)

def top_boundary_condition(K, T, S_max, r, t):
    return S_max - np.exp(-r * (T - t)) * K

def final_boundary_condition(K, T, S):
    return np.maximum(S - K, 0)

def sigma_t(t, T):
    return np.where(t > 0.40, 0.14 + 0.12 * (t - 0.40) / (T - 0.40),
0.14)

def compute_abc(K, T, t, sigma_t, r, S, dt, dS, gamma):
    sigma = sigma_t(t, T)
    a = -sigma**2 * S**(2 * gamma) / (2 * dS**2) + r * S / (2 * dS)
    b = r + sigma**2 * S**(2 * gamma) / (dS**2)
```

```
    c = -sigma**2 * S**(2 * gamma) / (2 * dS**2) - r * S / (2 * dS)
    return a, b, c

def compute_lambda(a, b, c):
    return scipy.sparse.diags([a[1:], b, c[:-1]], offsets=[-1, 0, 1],
format='csr')

def compute_W(a, b, c, V0, VM):
    W = np.zeros(len(b))
    W[0] = a[0] * V0
    W[-1] = c[-1] * VM
    return W

def price_call_crank_nicolson(K, T, r, sigma_t, N, M, gamma, S0):
    dt = T / N
    S_min = 0
    S_max = K * np.exp(8 * sigma_t(0, T) * np.sqrt(T))
    dS = (S_max - S_min) / M
    S = np.linspace(S_min, S_max, M + 1)
    t = np.linspace(0, T, N + 1)
    V = np.zeros((N + 1, M + 1))


    V[:, -1] = top_boundary_condition(K, T, S_max, r, t)
    V[:, 0] = bottom_boundary_condition(K, T, S_min, r, t)
    V[-1, :] = final_boundary_condition(K, T, S)

    for i in range(N - 1, -1, -1):
        a, b, c = compute_abc(K, T, t[i], sigma_t, r, S[1:-1], dt, dS,
gamma)
        Lambda = compute_lambda(a, b, c)
        identity = scipy.sparse.identity(M - 1)

        Wt = compute_W(a, b, c, V[i, 0], V[i, M])
        Wt_plus_dt = compute_W(a, b, c, V[i + 1, 0], V[i + 1, M])

        V[i, 1:M] = scipy.sparse.linalg.spsolve(
            identity + 0.5 * Lambda * dt,
            (identity - 0.5 * Lambda * dt).dot(V[i + 1, 1:M]) - 0.5 *
dt * (Wt_plus_dt + Wt)
        )


    price_at_S0 = np.interp(S0, S, V[0, :])

    return price_at_S0


M_list = np.array([44, 100, 223, 316, 707, 1000, 2236])


prices_crank = []
for M, N in zip(M_list, M_list):
    price_cn = price_call_crank_nicolson(K, T, r, sigma_t, N, M,
gamma, S0)
    prices_crank.append(price_cn)


x_values_crank = M_list * M_list
plt.figure(figsize=(10, 4))
plt.plot(x_values, prices, marker='o', linestyle='-',color='blue',
```

```
label="Monte Carlo")
plt.plot(x_values, prices_cv, marker='s', linestyle='-',color =
'green', label="Control Variate")
plt.plot(x_values_crank, prices_crank, marker='x', color
='black',linestyle='-', label="FDM")
price_estimate = 2.5850
plt.axhline(y=price_estimate, color='r', linestyle='--', label="True
Price (2.5850)")

plt.xscale("log")
plt.xlabel("Total Points (MC: Paths × 100, FDM: M × N)", fontsize=20)
plt.ylabel("Price of Option", fontsize=20)
plt.legend()
plt.grid(True, which="both", linestyle="--", linewidth=0.5)
plt.show()
```

Out[28]:



In [0]: